

Rozproszone systemy akwizycji danych - Laboratorium

Raport z wykonania ćwiczenia

Ćwiczenie nr: 1		Wykonano dnia: 26.10.2025 13:30	
Nazwa Ćwiczenia: System akwizycji danych - założenia, asembler mikrokontrolerów AVR			
Wykonawcy Ćwiczenia:			
Anton Shevchenko		Bartłomiej Walczak	

1. Programy (obowiązkowe)

Zadanie 1: Napisz podprogram odmierzający czas 104us. Zastosuj napisany podprogram w programie programowego nadajnika UART. Uwzględnij opóźnienia spowodowane przez rozkazy CALL i RET.

```
WAIT:
    ldi r16, 136      ; licznik pętli
L1:
    dec r16           ; 1 cykl
    brne L1           ; 2 cykle, gdy warunek prawdziwy (1 cykl, gdy
falszywy)
    ret               ; powrót (4 cykle)
```

Dla mikrokontrolera ATmega16 (przy zegarze 8 MHz) **1 cykl = 0,125 μs**. Z tego wynika, że **104 μs = 832 cykle**. Dodajemy do tego CALL + RET = 8 cykli. Licząc całość pętla w podprogramie musi trwać **824 cykle**.

Wywołanie programu za pomocą instrukcji `call WAIT`.

Zadanie 2: Napisz program, który przez port szeregowy wyśle twoje inicjały (dwa znaki w kodzie ASCII).

```
.nolist
#include "m16def.inc"
.list
.listmac
.device ATmega16

.cseg

.org 0x0000
jmp START

WAIT:
    ldi r16, 136
L1:
    dec r16
    brne L1
    ret

.org 0x0030
START:

    //stos
    ldi r16, 0x5F
    out SPL, r16
    ldi r17, 0x04
    out SPH, r17

    //port B - wyjście
    ldi r16, 0b00000001
    out DDRB, r16

    //Inicjaly AS

    //bit - start
    cbi PORTB,0
    call WAIT

    //'A'
    sbi PORTB,0 //1
    call WAIT

    cbi PORTB,0 //0
    call WAIT

    cbi PORTB,0 //0
    call WAIT
```

```
cbi PORTB,0 //0
call WAIT

cbi PORTB,0 //0
call WAIT //0

sbi PORTB,0 //1
call WAIT

cbi PORTB,0 //0
call WAIT

//bit - stop
cbi PORTB,0 //0
call WAIT


//bit - start
cbi PORTB,0
call WAIT

//'S'
sbi PORTB,0 //1
call WAIT

sbi PORTB,0 //1
call WAIT

cbi PORTB,0 //0
call WAIT

cbi PORTB,0 //0
call WAIT

sbi PORTB,0 //1
call WAIT

cbi PORTB,0 //0
call WAIT

sbi PORTB,0 //1
call WAIT

cbi PORTB,0 //0
call WAIT

//bit - stop
sbi PORTB,0 //1
call WAIT
```

```
jmp PC
```

cbi PORTB,0 - Ustawia linię TX = 0 (bit START)

sbi PORTB,0 - Ustawia linię TX = 1 (bit danych lub STOP)

call WAIT - Odmierza dokładny czas trwania jednego bitu (104 µs)

3. Zadania dodatkowe (dla chętnych)

Zadanie A (na ocenę 4.0): Napisz program, który wyśle przez UART (programowy) dowolny znak ASCII (dowolną zawartość rejestru r16). Napisz podprogramy wysyłania 0 i 1, wykorzystaj instrukcje przesuwania rejestru i testowania znacznika C.

Zadanie B (na ocenę 5.0): Bazując na programie z Zadania A, napisz program który wyśle przez port szeregowy (programowy) twoje Imię i Nazwisko.

```
.nolist
.include "m16def.inc"
.list
.listmac
.device ATmega16

.cseg

.org 0x0000
    jmp START

//104 us
WAIT:
    ldi r16, 136
L1:
    dec r16
    brne L1
    ret

//wysyłanie logicznego 0
SEND_ZERO:
    cbi PORTB,0
    call WAIT
    ret

//wysyłanie logicznej 1
SEND_ONE:
    sbi PORTB,0
    call WAIT
    ret
```

```
//podprogram TX_BYTE – wysyła jeden bajt
```

```
TX_BYTE:
```

```
    call SEND_ZERO
```

```
    ldi r17,8
```

```
TX_LOOP:
```

```
    lsr r16
```

```
    brcs SEND_ONE_BIT
```

```
    call SEND_ZERO
```

```
    rjmp NEXT_BIT
```

```
SEND_ONE_BIT:
```

```
    call SEND_ONE
```

```
NEXT_BIT:
```

```
    dec r17
```

```
    brne TX_LOOP
```

```
    call SEND_ONE
```

```
    ret
```

```
.org 0x0030
```

```
START:
```

```
    ldi r16,0x5F
```

```
    out SPL,r16
```

```
    ldi r17,0x04
```

```
    out SPH,r17
```

```
    ldi r16,0b00000001
```

```
    out DDRB,r16
```

```
    ldi r16,'A'
```

```
    call TX_BYTE
```

```
    ldi r16,'n'
```

```
    call TX_BYTE
```

```
    ldi r16,'t'
```

```
    call TX_BYTE
```

```
    ldi r16,'o'
```

```
    call TX_BYTE
```

```
    ldi r16,'n'
```

```
    call TX_BYTE
```

```
    ldi r16,' '
```

```
    call TX_BYTE
```

```
    ldi r16,'S'
```

```
    call TX_BYTE
```

```
    ldi r16,'h'
```

```
    call TX_BYTE
```

```
    ldi r16,'e'
```

```
    call TX_BYTE
```

```
    ldi r16,'v'
```

```
    call TX_BYTE
```

```
    ldi r16,'c'
```

```
    call TX_BYTE
```

```
    ldi r16,'h'
```

```
    call TX_BYTE
```

```
ldi r16, 'e'
call TX_BYTE
ldi r16, 'n'
call TX_BYTE
ldi r16, 'k'
call TX_BYTE
ldi r16, 'o'
call TX_BYTE

rjmp PC
```

Program działa tak, że najpierw mikrokontroler ustawia sobie stos i pin PB0 jako wyjście, ponieważ przez niego będą wysyłane dane. Następnie po kolei wysyła litery imienia i nazwiska „Anton Shevchenko”. Każda litera w kodzie ASCII jest przesyłana bit po bicie. Na początku każdego znaku pojawia się bit startu, czyli linia PB0 jest ustawiona na 0, aby odbiornik wiedział, że zaraz zacznie się nowy znak. Potem wysyłane są 8 bitów danych, zaczynając od najmłodszego bitu, a na końcu pojawia się bit stopu, który jest jedynką. Dzięki podprogramowi WAIT każdy bit trwa około 104 mikrosekund, co odpowiada prędkości transmisji 9600 bps. Po wysłaniu wszystkich liter program zatrzymuje się w nieskończonej pętli. Po podłączeniu mikrokontrolera do komputera lub konwertera USB-UART na terminalu pojawia się napis „Anton Shevchenko”.